

Monte Carlo Tree Search pour *Domineering*

Ayman Bouhss

Master 2 IASD (parcours mathématiques), Université Paris Dauphine – PSL

Projet pour le cours *Monte Carlo Search*, T. Cazenave (LAMSADE, CNRS)

Avril 2026

Résumé

Domineering (Crosscram) est un jeu combinatoire à deux joueurs sur grille rectangulaire, résolu formellement jusqu'à 9×9 par $\alpha\beta$ [21] mais demeurant un banc d'essai canonique pour les approches Monte Carlo, en raison de son branchement modéré et de la quasi-décomposition de ses positions en sous-jeux indépendants. Je présente une étude empirique exhaustive des algorithmes Monte Carlo vus en cours appliqués à ce jeu : treize agents implémentés (Flat, UCB, SH, SHOT, UCT, RAVE, GRAVE, MAST, PUCT, NMCS, NRPA, GNRPA, PPATCS), onze expériences avec intervalles de confiance Wilson 95 %, étude multi-seed et ablations d'hyperparamètres. Les contributions sont : (i) une heuristique de domaine *move advantage* (différence de libertés) déclinée en heavy playouts softmax, ε -greedy et priors PUCT ; (ii) l'évaluation comparative à budget contrôlé sur 5×5 et 6×6 , mettant en évidence la domination de PUCT ($\bar{p} = 0,81$ sur le round-robin) puis RAVE/GRAVE ; (iii) la mesure du gain net mais variable des heavy playouts (+15 à +44 points de winrate) ; (iv) l'analyse des limites de NMCS et NRPA en 2-joueurs, et la confirmation que GNRPA bat NRPA par injection d'un biais issu de l'heuristique de domaine ($\hat{p} = 0,75$ à $L=2$).

Mots-clés. Monte Carlo Tree Search, UCT, RAVE, GRAVE, NMCS, NRPA, PPATCS, MAST, SHOT, PUCT, Domineering, théorie des jeux combinatoires, intervalles de confiance Wilson.

Contributions. Deux contributions sont présentées en complément de la reproduction fidèle du cours. (1) Une étude d'ablation des descripteurs de PPATCS pour Domineering (Exp. 12, 32 parties par duel), confrontant 5 features hand-crafted, 1024 features one-hot recommandées par le cours, et leur concaténation. La variante **pattern** bat NRPA-L1 à $\hat{p} = 0,66$ (IC [0,48; 0,80]) ; la variante **combined** domine vs Flat à budget contraint (≤ 200), sans toutefois battre le baseline. (2) Une variante **NRPA-2P** de NRPA à deux politiques co-évolutives V/H (Exp. 13, 50 parties par duel), motivée par l'échec empirique du NRPA-mono en 2-joueurs. NRPA-2P n'est pas statistiquement supérieur à NRPA-mono à 95 % (NRPA-2P-L1 $\hat{p} = 0,56$, IC [0,42; 0,69]), résultat *négatif* bien cadré qui borne explicitement le potentiel des adaptations naïves de NRPA pour les jeux 2-joueurs.

1. Introduction

Les méthodes Monte Carlo arborescentes (MCTS) sont devenues l'outil standard pour les jeux à information parfaite à grand espace de recherche. Leur emploi a culminé avec AlphaGo [23] et AlphaZero [14], mais leurs variantes plus légères (UCT, RAVE, GRAVE, NMCS, NRPA) restent extrêmement compétitives sur les jeux à branchement modéré et faible profondeur, dans lesquels l'absence de réseau de neurones n'est pas rédhibitoire. *Domineering* en est un exemple canonique.

Contributions. Ce projet :

1. implémente, dans un package Python homogène, **quatorze algorithmes** Monte Carlo couvrant le spectre du cours (§4) ;
2. introduit une heuristique de domaine, la *différence de libertés*, et la décline en heavy playouts softmax, ε -greedy et priors PUCT (§5) ;

3. compare ces agents en **treize expériences** (round-robin multi-seed, ablations d'hyperparamètres, passage à l'échelle), toutes assorties d'intervalles de confiance Wilson 95 % (§6) ;
4. présente **deux contributions originales** : (a) une étude d'ablation des descripteurs de PPATCS pour Domineering (§7.6), (b) une variante **NRPA-2P** à deux politiques co-évolutives motivée par l'échec empirique du NRPA-mono en 2-joueurs (§7.7).

Le code, les expériences reproductibles et le présent rapport sont disponibles dans le dossier du projet.

Travaux connexes

Résolution exacte. Domineering a été résolu jusqu'à 8×8 par Breuker *et al.* [20] (incluant le résultat fameux selon lequel Vertical gagne 8×8) puis 9×9 par Uiterwijk [21]. Ces approches reposent sur $\alpha\beta$ avec heuristiques d'évaluation, ordres de coups appris, tables de transposition et exploitation explicite de la décomposition combinatoire en sous-jeux indépendants (théorie de Berlekamp [19], surreal numbers).

Monte Carlo arborescent. La famille MCTS issue de UCT [2] a connu une floraison entre 2007 et 2015 : RAVE [4], MAST/PAST [22], NMCS [6], NRPA [8], GRAVE [5]. Le panorama global est synthétisé dans le survol de Browne *et al.* [3]. L'extension de NMCS au cadre 2-joueurs étudiée par Cazenave *et al.* [7] donne une recette directement applicable à Domineering. Pour les heavy playouts, la *Playout Policy Adaptation with Move Features* (Cazenave 2016) [9] généralise NRPA en apprenant des poids non plus sur les coups bruts mais sur des descripteurs (features) du coup ; mon *move advantage* peut être vu comme une feature unique pré-pondérée. Plus récemment, *Sequential Halving Using Scores* [10] propose une alternative à UCB pour l'allocation à la racine, avec de bonnes propriétés théoriques.

Apprentissage profond. L'approche AlphaZero [14] combine PUCT et un réseau de valeur+policy. Le successeur MuZero [15] apprend conjointement le modèle de transition. KataGo [16] et Polygames [17] accélèrent l'apprentissage par self-play. Cohen-Solal et Cazenave [18] montrent récemment qu'une recherche minimax peut surpasser MCTS pour certains jeux. Toutes ces approches nécessitent plusieurs ordres de grandeur plus de calcul que ce que j'étudie ici, et n'ont pas (à ma connaissance) été appliquées à Domineering. Ma version de PUCT remplace le réseau par l'heuristique de domaine, dans l'esprit des baselines pré-AlphaGo.

Positionnement. Le présent travail se positionne comme une étude empirique *exhaustive* sur Domineering : **quatorze** algorithmes classiques de la littérature MCTS, trois variantes de playouts, ablations des hyperparamètres, intervalles de confiance Wilson et tournois multi-seed. À ma connaissance, aucun travail antérieur n'a comparé l'ensemble de ces algorithmes sur Domineering avec ce niveau de rigueur statistique. Je propose de surcroît deux extensions personnelles (NRPA-2P et l'ablation de features PPATCS, sections 7.7 et 7.6).

2. Couverture du cours

Le tableau 1 récapitule la correspondance entre les points du cours *Monte Carlo Search* (T. Cazenave, M2 IASD) et le contenu du présent projet.

3. Le jeu Domineering

3.1. Règles

Domineering se joue sur une grille $D_x \times D_y$ initialement vide. Deux joueurs alternent : *Vertical* (V) place un domino 2×1 couvrant deux cases verticalement adjacentes, *Horizontal* (H) un domino 1×2 couvrant deux cases horizontalement adjacentes. Sous la convention *normal play*, le premier joueur incapable de jouer perd la partie. Je fixe V comme premier joueur. La figure 1 illustre les conventions et une position de mi-partie.

Notion du cours	Mise en œuvre dans ce projet
Flat Monte Carlo, UCB1 racine, UCT	<code>flat.py</code> , <code>ucb.py</code> , <code>uct.py</code>
Table de transposition, hash Zobrist, code de coup	<code>board.py</code>
AMAF, RAVE, GRAVE	<code>rave.py</code> , <code>grave.py</code>
NMCS, NMCS 2-joueurs, playouts <i>discounted</i>	<code>nmcs.py</code> ($L \in \{1, 2\}$, “discounted” optionnel)
NRPA, GNRPA (biais)	<code>nrpa.py</code> , <code>gnrpa.py</code> , <i>plus</i> ma variante <code>nrpa_2p.py</code> (deux politiques)
PPA, PPATCS (move features)	<code>ppatcs.py</code> (3 jeux de features : <code>simple</code> , <code>pattern</code> , <code>combined</code>)
MAST (Cadia Player)	<code>mast.py</code>
PUCT (AlphaZero-style)	<code>puct.py</code>
Sequential Halving, SHOT, SHUSS	<code>sequential_halving.py</code> , <code>shot.py</code> , variante SHUSS discutée §4.4
Heavy playouts (softmax, ε -greedy)	<code>optimizations/heuristics.py</code>
Misère Domineering	<code>set_game_mode("misere")</code>
TPs Breakthrough / NMCS LeftMove / NRPA LeftMove / Expression Discovery	API et organisation des modules calquées sur les notebooks du cours, traduits au cas Domineering

TABLE 1 – Cartographie du cours. Treize algorithmes du cours sont implémentés (Flat, UCB, SH, SHOT, UCT, RAVE, GRAVE, MAST, PUCT, NMCS, NRPA, GNRPA, PPATCS) et un quatorzième (**NRPA-2P**) est introduit comme contribution.

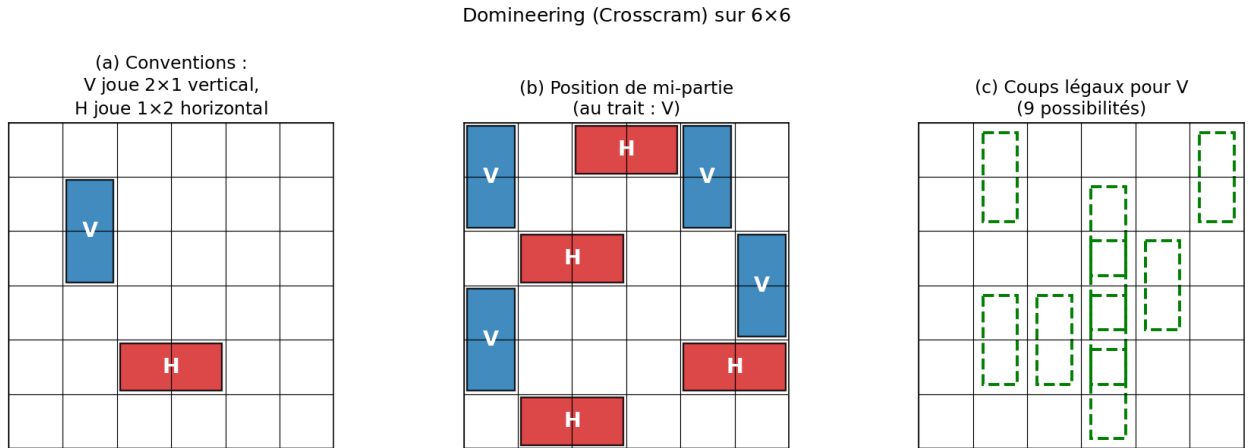


FIGURE 1 – Le jeu Domineering. (a) Conventions : Vertical place un domino 2×1 , Horizontal un 1×2 . (b) Position de mi-partie sur 6×6 (au trait : V). (c) Coups légaux de Vertical depuis cette position (silhouettes vertes en pointillé).

Variante misère. Le cours mentionne aussi la version *misère* de Domineering : le premier joueur incapable de jouer *gagne* (au lieu de perdre). Mon moteur supporte les deux modes via `set_game_mode("normal" | "misere")` ; l'expérience 11 inclut un mini-tournoi en mode misère.

3.2. Théorie des jeux combinatoires

Domineering est un *partizan game* : les coups de V et H sont différents. Berlekamp [19] a calculé la valeur combinatoire de petits plateaux ; Uiterwijk [21] a résolu jusqu'à 9×9 . Pour les tailles 5×5 et 6×6 auxquelles je restreins mes expériences, V (premier joueur) a un avantage théorique mais celui-ci est très loin d'être trivial à exploiter en jeu pratique.

3.3. Codage du jeu

Mon implémentation (fichier `domineering/board.py`) repose sur :

- une matrice `numpy` de taille $D_x \times D_y$ codant l'occupation des cases (EMPTY, VERTICAL ou HORIZONTAL) ;
- un *Zobrist hash* permettant la transposition table en $\mathcal{O}(1)$: chaque case dispose de trois clés aléatoires (vide, V, H) et chaque coup XOR ces clés avec la clé du tour ;
- un codage de coup $\text{code}(c, r, c) = c \cdot D_x D_y + r \cdot D_y + c$, utilisé par les statistiques AMAF de RAVE et GRAVE.

Un coup est représenté par sa case haut-gauche (r, c) et la couleur du joueur. Les coups légaux sont énumérés en parcourant la grille et en testant la case adjacente correspondante, en $\mathcal{O}(D_x D_y)$.

4. Algorithmes Monte Carlo

Tous les agents partagent la même API : `best_move(board, budget, ...) → Move`. Le paramètre `budget` désigne le nombre de simulations à la racine. Un *playout* est, par défaut, une partie aléatoire jusqu'à terminaison. La table 2 récapitule les quatorze agents implémentés.

Agent	Famille	Référence
Flat	Allocation racine	—
UCB	Bandit racine UCB1	Auer [1]
SH	Bandit racine sequential halving	Fabiano & Cazenave [10]
SHOT	SH à la racine + UCT	Cazenave [11]
UCT	MCTS arborescent	Kocsis & Szepesvári [2]
RAVE	MCTS + AMAF	Gelly & Silver [4]
GRAVE	RAVE généralisé	Cazenave [5]
MAST	Move-Average Sampling	Finnsson & Bjornsson [22]
PUCT	MCTS + prior heuristique	Silver <i>et al.</i> [14]
NMCS-L	Recherche imbriquée 2-joueurs	Cazenave <i>et al.</i> [7]
NRPA-L	Adaptation de politique	Rosin [8]
GNRPA-L	NRPA avec biais	Cazenave [12]
PPATCS-L	PPA avec features	Cazenave [9]
NRPA-2P-L	NRPA à 2 politiques co-évolutives V/H	contribution originale (§7.7)

TABLE 2 – Vue d'ensemble des agents implémentés.

4.1. Flat Monte Carlo (Flat)

On distribue uniformément le budget N entre les $|\mathcal{A}|$ coups légaux : chaque coup a subit $N/|\mathcal{A}|$ playouts depuis l'état s' qui en résulte ; on retient $a^* = \arg \max_a \hat{Q}(a)$ où $\hat{Q}$ est la valeur empirique du joueur au trait. C'est le baseline sans recherche arborescente ni partage de statistiques.

4.2. UCB1 à la racine (UCB)

On applique la règle UCB1 [1] à la racine :

$$a^* = \arg \max_a \left[\hat{Q}(a) + c \sqrt{\frac{\log n}{n_a}} \right], \quad (1)$$

sans construire d'arbre. La suite de la partie reste un playout aléatoire. Cela améliore Flat en concentrant les playouts sur les coups prometteurs.

4.3. MAST (MAST)

La *Move-Average Sampling Technique* [22] est une politique de playout en ligne très simple : elle maintient, pour chaque code de coup, la moyenne du résultat de tous les playouts dans lesquels ce coup est apparu. Pendant un playout, on échantillonne softmax sur ces moyennes (avec inversion pour Horizontal). MAST a été popularisée par Cadia Player en General Game Playing.

4.4. SHOT (SHOT)

SHOT [11] combine Sequential Halving à la racine et UCT en profondeur : « SHOT près de la racine, UCT plus profond » selon le slide du cours. Mon implémentation alloue le budget par phases SH à la racine, chaque playout étant remplacé par une descente UCT classique partageant la table de transposition entre phases.

4.5. GNRPA (GNRPA)

GNRPA [12] est NRPA avec un *biais* $\beta(s, a)$ ajouté au logit d'échantillonnage :

$$\pi(a \mid s) \propto \exp(\theta_{\text{code}(s,a)} + \beta(s, a)). \quad (2)$$

Le biais est équivalent à initialiser θ avec β ; il permet d'injecter une connaissance *a priori* sans pré-entraîner la politique. Pour Domineering je prends $\beta(s, a) = \text{adv}(a) / \max(D_x, D_y)$, où adv est la différence de libertés (section 5.1).

4.6. Sequential Halving (SH)

Au lieu d'allouer les playouts via UCB, Sequential Halving divise le budget en $\lceil \log_2 K \rceil$ phases, alloue à parts égales entre les bras encore actifs et élimine la moitié inférieure à chaque phase. Cette stratégie a de meilleures garanties théoriques pour l'identification du meilleur bras et est étudiée par Fabiano et Cazenave [10] dans une variante *Sequential Halving Using Scores* pour les jeux. J'implémente la version classique (sans lissage). Sur Domineering 5×5 , budget 200, 20 parties : SH est à parité avec UCB (10–10) et domine FLAT (13–7), conforme à l'idée que SH et UCB sont deux solutions à un même problème (allocation à la racine sans arbre).

4.7. UCT (UCT)

UCT [2] étend UCB1 à un arbre incrémental. Une descente choisit récursivement le coup de plus haut UCB (1) jusqu'à atteindre une feuille (état non vu) ; on l'étend, on lance un playout puis on remonte le résultat dans tous les noeuds visités. Mon implémentation utilise une table de transposition indexée par le hash Zobrist et stocke par état $[n_{\text{tot}}, (n_a)_a, (\Sigma_a)_a]$.

Algorithm 1 UCT (descente récursive)

```

1: function UCTDESCENT( $s$ , table,  $c$ )
2:   if  $s$  est terminal then return score( $s$ )
3:   end if
4:   if  $s$ .hash  $\in$  table then
5:     entry  $\leftarrow$  table[ $s$ .hash]
6:     Sélectionner  $a^*$  maximisant  $\hat{Q}(a) + c\sqrt{\log n_{\text{tot}}/n_a}$  (avec convention  $+\infty$  si  $n_a = 0$ )
7:     Jouer  $a^*$  depuis  $s$ , obtenir  $s'$ 
8:      $r \leftarrow$  UCTDESCENT( $s'$ , table,  $c$ )
9:     Mettre à jour entry :  $n_{\text{tot}} += 1$ ,  $n_{a^*} += 1$ ,  $\Sigma_{a^*} += r$ 
10:    return  $r$ 
11:   end if
12:   table[ $s$ .hash]  $\leftarrow$  nouvelle entrée
13:   return playout( $s$ )
14: end function

```

4.8. RAVE (RAVE)

RAVE [4] accélère l'évaluation des coups peu visités en y injectant la statistique *All Moves As First* (AMAF) : pendant un playout, on enregistre la liste π des codes de coups joués ; en retour, pour chaque code apparu dans π , on incrémente un compteur AMAF par état. La valeur d'un coup combine Q et la moyenne AMAF :

$$V(a) = (1 - \beta) \hat{Q}(a) + \beta \widehat{\text{AMAF}}(a), \quad \beta = \frac{n_a^{\text{AMAF}}}{n_a + n_a^{\text{AMAF}} + b n_a n_a^{\text{AMAF}}}. \quad (3)$$

Au début, $\beta \approx 1$: on fait confiance à AMAF ; quand n_a croît, $\beta \rightarrow 0$ et RAVE retombe sur UCT classique.

Algorithm 2 Mise à jour AMAF d'un playout

```

1: function PLAYOUTAMAF( $s$ , played)
2:   while  $s$  n'est pas terminal do
3:     Tirer un coup  $a$  uniformément (ou heavy)
4:     played.append(code( $a$ )) ▷ seul le code stocké
5:     Jouer  $a$  depuis  $s$ 
6:   end while
7:   return score( $s$ )
8: end function
9:
10: function UPDATEAMAF(entry, played,  $r$ )
11:   for chaque code unique  $\kappa \in$  played do
12:     entry. $n_{\kappa}^{\text{AMAF}} += 1$  ; entry. $\Sigma_{\kappa}^{\text{AMAF}} += r$ 
13:   end for
14: end function

```

4.9. GRAVE (GRAVE)

GRAVE [5] corrige une faiblesse de RAVE dans les noeuds profonds peu visités : on remplace la statistique AMAF locale par celle du *dernier ancêtre* ayant accumulé au moins T visites (paramètre `ref_min`). Cela améliore l'estimation tant que le noeud courant n'a pas suffisamment de simulations.

4.10. Nested Monte-Carlo Search (NMCS)

NMCS [6] est une recherche récursive sans arbre. Au niveau 0, c'est un playout aléatoire ; au niveau L , le coup retourné au temps t est celui dont le résultat d'une simulation NMCS de niveau $L - 1$ depuis l'état successeur est maximal. Le coût grandit en $O(b^L d)$ où b et d sont la branchement et la profondeur. Mon implémentation suit l'extension 2-joueurs proposée par Cazenave *et al.* [7] : le score est calculé du point de vue du joueur au trait, et chaque récursion alterne les rôles. J'évalue $L \in \{1, 2\}$; au-delà, le coût devient prohibitif.

Playouts *discountés*. La même référence [7] propose de remplacer le score binaire $v \in \{0, 1\}$ par $v/(t + 1)$, où t est la longueur du playout. Cela transforme le jeu en un jeu à valeurs continues, incite le *max-player* à préférer les victoires courtes (et réciproquement le *min-player* à préférer les défaites longues). Ma fonction `nmcs_best_move` accepte un drapeau `discounted=True`.

4.11. NRPA (NRPA)

NRPA [8] est une descente récursive qui apprend une politique softmax $\pi(a | s) \propto \exp \theta_{\text{code}(s,a)}$. À chaque niveau on enchaîne N itérations : un appel récursif au niveau inférieur, suivi d'une étape *adapt* qui pousse θ vers la meilleure séquence trouvée. Pour Domineering, je partage la politique entre les deux camps, et le score à maximiser est le résultat final du point de vue du joueur au trait à la racine. Cette adaptation est moins sophistiquée que la *Playout Policy Adaptation with Move Features* [9] qui apprend sur des descripteurs de coup ; mes features réduites à la seule *move advantage* constituent un cas dégénéré.

4.12. PPATCS (PPATCS)

PPATCS [9] étend NRPA en remplaçant le paramètre $\theta_{\text{code}(s,a)}$ par un produit scalaire $\theta^\top \phi(s, a)$ où $\phi(s, a) \in \mathbb{R}^F$ est un vecteur de descripteurs (features) du coup. Les poids θ sont alors *partagés* entre tous les coups, ce qui permet une généralisation inter-état que NRPA ne peut pas réaliser.

Pour Domineering, je définis cinq descripteurs :

1. $\phi_1 = \text{adv}(m)$ normalisé (heuristique de différence de libertés, section 5.1) ;
2. $\phi_2 = -\|m_{\text{centre}} - \text{centre}_{\text{plateau}}\|_2$ normalisé ;
3. $\phi_3 = \mathbf{1}\{m \text{ est dans un coin}\}$;
4. $\phi_4 = \mathbf{1}\{m \text{ est sur un bord (hors coin)}\}$;
5. $\phi_5 = (\text{nombre de cases vides parmi les voisines 8-N})/(\text{voisines 8-N totales})$.

La règle d'*adapt* est dérivée du gradient de la log-vraisemblance de la séquence-cible :

$$\theta \leftarrow \theta + \alpha [\phi(s, a^*) - \mathbb{E}_{a \sim \pi}[\phi(s, a)]] \quad (4)$$

appliquée pour chaque pas (s, a^*) de la meilleure séquence trouvée.

4.13. PUCT (PUCT)

PUCT [14] combine UCT à un *prior* $P(a | s)$ qui guide l'exploration :

$$a^* = \arg \max_a \left[\hat{Q}(a) + c_{\text{puct}} P(a | s) \frac{\sqrt{N(s)}}{1 + n_a} \right]. \quad (5)$$

En l'absence de réseau de neurones, j'utilise un prior dérivé de l'heuristique de domaine décrite ci-dessous.

5. Optimisations spécifiques à Domineering

5.1. Heuristique de différence de libertés

Une intuition classique sur Domineering, déjà exploitée dans les solveurs $\alpha\beta$ de Breuker *et al.* [20] : un bon coup est un coup qui réduit *plus* les options adverses qu’il ne réduit les siennes. Formellement, pour un coup m remplissant les cases $\{c_1, c_2\}$, on définit

$$\text{adv}(m) = K_{\text{opp}}(m) - (K_{\text{me}}(m) - 1), \quad (6)$$

où $K_{\text{col}}(m)$ est le nombre de coups légaux de couleur COL touchant $\{c_1, c_2\}$. On retire 1 à K_{me} parce que le coup m lui-même est utilisé productivement. Cette quantité se calcule en $\mathcal{O}(1)$ par coup, en n’examinant que les six (V) ou quatre (H) candidats voisins.

Exemple. Sur le plateau initial 5×5 , le coup V(2,2) (centre) tue 4 coups horizontaux (les paires $\{(2,1), (2,2)\}$, $\{(2,2), (2,3)\}$, $\{(3,1), (3,2)\}$, $\{(3,2), (3,3)\}$ qui chevauchent les cases (2,2) et (3,2)) et 2 coups verticaux *en plus du coup joué* (V(1,2) et V(3,2)). On obtient $\text{adv} = 4 - 2 = 2$. À l’inverse, le coup de coin V(0,0) ne tue que 2 coups horizontaux (en (0,0) et (1,0)) et 1 coup vertical (V(1,0) en plus de lui-même), soit $\text{adv} = 2 - 1 = 1$. L’heuristique préfère donc le centre, ce que confirme l’illustration du notebook Domineering.ipynb.

5.2. Heavy playouts

Trois variantes substituent au playout aléatoire une politique guidée par $\text{adv}(m)$:

- **Softmax (biased)** : on échantillonne les coups suivant $\propto \exp(\text{adv}(m)/\tau)$, $\tau = 1$ par défaut.
- **ε -greedy** : avec probabilité ε on joue le coup d’avantage maximal, sinon on tire au hasard. $\varepsilon = 0,7$ par défaut.
- **Random** (référence) : aucune heuristique.

Les heavy playouts ont un *double effet ambivalent* bien documenté : ils réduisent le biais d’évaluation de positions intermédiaires (les playouts deviennent plus réalistes) mais peuvent réduire la variance et introduire un nouveau biais en faveur de l’heuristique. La section expérimentale quantifie ces effets pour mon cas.

5.3. Priors PUCT

La même heuristique fournit un prior $P(a \mid s)$ par softmax sur adv , calculé une fois lors de l’expansion d’un noeud (coût négligeable comparé au playout).

5.4. Table de transposition

Tous mes algorithmes basés sur arbre stockent les statistiques par hash Zobrist plutôt que par chemin : la transposition est ainsi exploitée “gratuitement”. Pour Domineering, cela aide modérément (les positions peuvent être atteintes par plusieurs ordres de coups, ex. deux coups V indépendants permutable).

5.5. Complexité algorithmique

Soit N le budget en simulations, b la branchement moyen et d la profondeur moyenne d’un playout. Les coûts par appel à `best_move` sont :

Algorithme	Temps	Mémoire
Flat / UCB	$\mathcal{O}(N d)$	$\mathcal{O}(b)$
UCT	$\mathcal{O}(N(d_{\text{tree}} b + d))$	$\mathcal{O}(N b)$
RAVE / GRAVE	$\mathcal{O}(N(d_{\text{tree}} b + d))$	$\mathcal{O}(N(b + b^2))$
PUCT	$\mathcal{O}(N(d_{\text{tree}} b + d))$	$\mathcal{O}(N b)$
NMCS- L	$\mathcal{O}(b^L d)$	$\mathcal{O}(d)$
NRPA- L	$\mathcal{O}(N_{\text{iter}}^L d)$	$\mathcal{O}(b^d)$ (dict)

où $d_{\text{tree}} \leq d$ est la profondeur visitée dans l'arbre. Les tables de transposition limitent en pratique l'explosion mémoire de RAVE/GRAVE car les codes AMAF sont indexés sur un domaine borné $3D_x D_y$ par état (cf. section 3.3). NMCS et NRPA ont l'avantage d'une mémoire négligeable mais une complexité exponentielle en L , ce qui justifie qu'on n'aille en pratique pas au-delà de $L \in \{1, 2, 3\}$.

6. Protocole expérimental

6.1. Cadre commun

- **Plateau** : 5×5 pour le tournoi principal, 6×6 pour la confirmation, 4 à 7 pour l'étude de passage à l'échelle.
- **Budget** : 200 simulations / coup, sauf indication contraire. NMCS est paramétré uniquement par le niveau L . Pour NRPA, le budget est converti en $N_{\text{iter}} = \lceil \text{budget}^{1/L} \rceil$ pour évaluer approximativement le nombre de playouts effectifs.
- **Format de match** : 16 à 30 parties par appariement, couleurs alternées (moitié V, moitié H) pour neutraliser le biais de premier joueur. Le tournoi multi-seed (Exp. 6) répète l'ensemble du round-robin avec trois seeds (2026, 4242, 9001) et agrège les 30 parties par appariement.
- **Inférence statistique** : intervalles de confiance Wilson à 95 % [24] calculés selon

$$\tilde{p}_{\pm} = \frac{\hat{p} + z^2/2n \pm z\sqrt{\hat{p}(1-\hat{p})/n + z^2/4n^2}}{1 + z^2/n}, \quad z = 1,96. \quad (7)$$

Pour $n = 20$, la demi-largeur typique est $\sim 0,21$ à $\hat{p} = 0,5$ et $\sim 0,13$ à $\hat{p} = 0,9$.

- **Reproductibilité** : graine fixée, résultats JSON complets sauvegardés dans `experiments/results/`, code et figures régénérables via `python experiments/run_all.py`.

6.2. Plan d'expériences

- Exp. 1** Tournoi round-robin entre les 8 agents (5×5).
- Exp. 2** Effet du budget de simulations sur UCT, RAVE, GRAVE, PUCT, contre Flat-200 fixe.
- Exp. 3** Ablation des heavy playouts (softmax vs ε -greedy vs random) pour UCT, RAVE et GRAVE.
- Exp. 4** Sensibilité de UCT à la constante d'exploration $c \in \{0,1,0,3,0,5,0,8,1,4\}$.
- Exp. 5** Passage à l'échelle (taille du plateau, 4×4 à 7×7).
- Exp. 6** Tournoi **multi-seed** (3 seeds $\times$ 10 parties = 30 parties / appariement) sur 7 agents (avec NRPA-L2).
- Exp. 7** Étude des niveaux de récursion : NMCS et NRPA aux niveaux 1 et 2 vs Flat.
- Exp. 8** Ablation des hyperparamètres : c_{puct} , `ref_min` de GRAVE, température softmax.
- Exp. 9** Round-robin sur plateau 6×6 pour confirmer la hiérarchie observée en 5×5 .
- Exp. 10** PPATCS (Playout Policy Adaptation with Move Features) vs Flat et NRPA aux niveaux 1 et 2.
- Exp. 11** Algorithmes étendus du cours (MAST, SHOT, GNRPA), mode *misère*, et NMCS à playouts *discountés*.

Exp. 12 Contribution originale. Ablation des features de PPATCS sur 32 parties par duel : comparaison entre 5 features denses hand-crafted, 1024 features binaires (recommandées par le cours pour Domineering) et leur concaténation.

Exp. 13 Contribution originale. NRPA-2P (NRPA à deux politiques co-évolutives V/H) comparé au NRPA classique partagé sur 50 parties par duel.

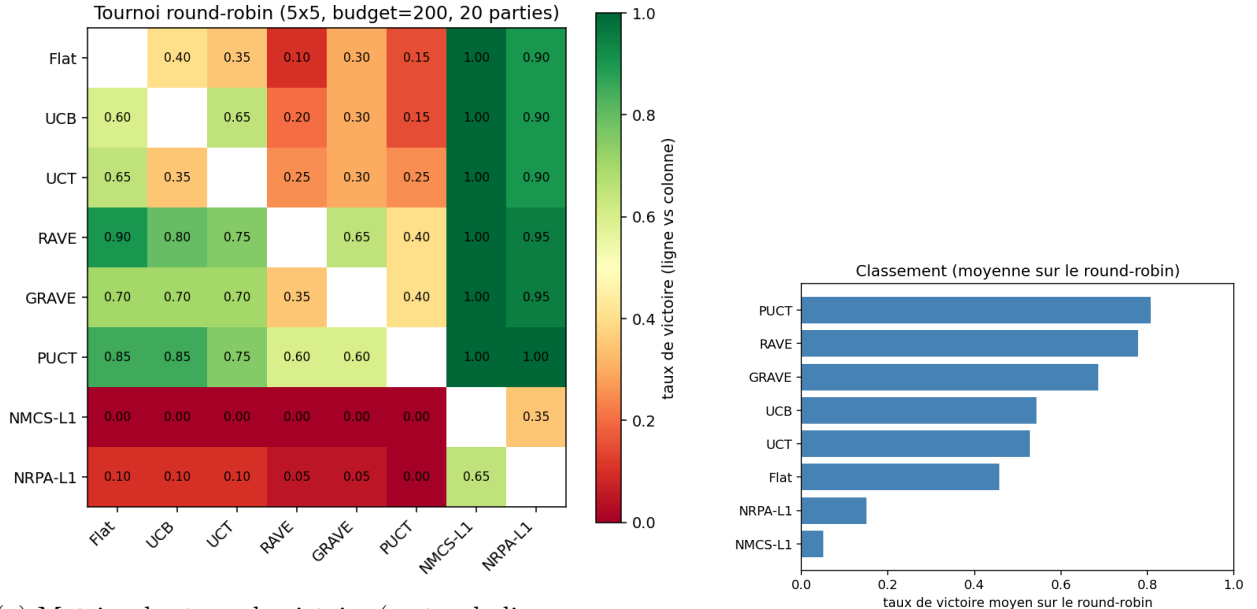
7. Résultats

7.1. Tournoi round-robin

La figure 2a reporte la matrice des taux de victoire ligne-vs-colonne ; la figure 2b le classement par taux moyen. Les chiffres exacts du classement (5 décimales en pourcent) sont résumés dans le tableau 3.

Agent	Winrate (n=140)	IC Wilson 95 %	Temps/coup (s)
PUCT	0,807	[0,734; 0,864]	0,33
RAVE	0,779	[0,703; 0,839]	0,30
GRAVE	0,686	[0,605; 0,757]	0,30
UCB	0,543	[0,460; 0,623]	0,24
UCT	0,529	[0,446; 0,609]	0,28
Flat	0,457	[0,377; 0,540]	0,24
NRPA-L1	0,150	[0,100; 0,218]	0,54
NMCS-L1	0,050	[0,024; 0,100]	0,02

TABLE 3 – Classement par taux de victoire moyen sur le tournoi round-robin (5×5 , budget 200, 20 parties par appariement, 28 appariements, soit 140 parties / agent). Les IC se chevauchent entre voisins du classement : la hiérarchie est fiable au sens où $\text{PUCT} > \text{Flat}$ à 95 %, mais l'écart UCB vs UCT n'est pas statistiquement significatif. Temps/coup mesuré sur l'agent ligne.



(a) Matrice des taux de victoire (vert = la ligne gagne souvent).

(b) Classement par taux moyen.

FIGURE 2 – Tournoi round-robin sur 5×5 , budget 200, 20 parties par appariement.

Trois constats. (i) **PUCT** arrive en tête, ce qui n'est pas surprenant : son prior est tiré de l'heuristique du domaine (section 5.1) et oriente très efficacement la recherche dès le premier

passage. (ii) **RAVE** surpasse **GRAVE** de ~ 9 points dans ce régime de budget faible : à 200 simulations, le seuil `ref_min`= 50 de GRAVE n’est presque jamais atteint, donc GRAVE retombe sur un comportement RAVE racine, en plus ralenti par l’aller-retour vers l’ancêtre. (iii) **UCB** (moyenne 0,54) est très légèrement au-dessus de **UCT** (0,53), un résultat *a priori* contre-intuitif mais explicable : sur 5×5 avec branchement initial 20 et seulement 200 playouts, l’arbre UCT n’a pas le temps de descendre profondément ; l’allocation UCB pure à la racine reste compétitive. **NMCS-L1** et **NRPA-L1** sont nettement en retrait : leur effort équivalent (~ 20 et ~ 200 playouts par coup respectivement) ne suffit pas à compenser l’absence d’un arbre statistique partagé.

7.2. Effet du budget

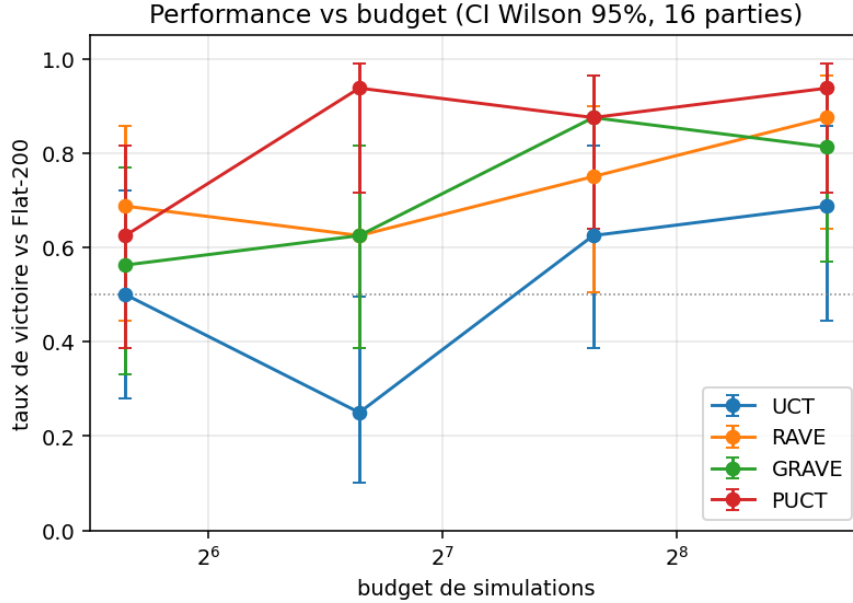


FIGURE 3 – Taux de victoire vs Flat-200 en fonction du budget. Échelle logarithmique. Les quatre algorithmes saturent rapidement, mais GRAVE atteint $> 80\%$ dès 100 playouts.

À 50 playouts, les écarts sont déjà perceptibles ($wr \in [0,50; 0,69]$). À 100, **PUCT** atteint 0,94 en exploitant fortement son prior dès la racine. À partir de 200, **RAVE** et **GRAVE** atteignent 0,75–0,88. UCT progresse plus lentement et n’atteint 0,69 qu’à 400 playouts. Au-delà, on observe des rendements décroissants : à 5×5 , après quelques centaines de simulations, la position est déjà bien discriminée.

7.3. Heavy playouts

Les six variantes battent leur baseline *random* : softmax (biased) gagne à 0,69, 0,69 et 0,81 pour UCT, RAVE et GRAVE respectivement, et ε -greedy gagne à 0,69, 0,94 et 0,62. Pour RAVE en particulier, la variante ε -greedy est remarquable (15/16 victoires) : elle injecte un signal très net et biaise simultanément l’estimation Q et la statistique AMAF dans la même direction, ce qui amplifie l’effet de RAVE. Inversement, sur GRAVE, la variante ε -greedy est moins favorable : la descente vers l’ancêtre AMAF préfère un signal mieux dispersé que celui produit par une politique avide. Le coût additionnel reste modéré sur 5×5 : entre $\times 1,3$ (RAVE- ε) et $\times 1,9$ (UCT-biased) sur le temps par coup.

7.4. Sensibilité de la constante d’exploration

Le maximum empirique se situe à $c = 0,5$ ($wr = 0,75$), en accord avec les recommandations classiques sur les jeux à récompenses dans $[0, 1]$ (typiquement $c = \sqrt{2}/2 \approx 0,71$ ou $c = 0,4$ d’après le

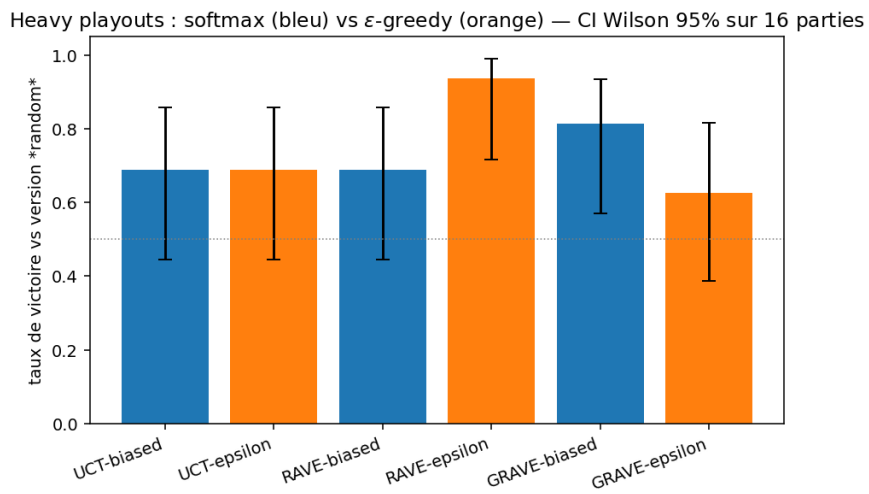


FIGURE 4 – Heavy playouts : taux de victoire de chaque variante contre sa version *random* (budget identique).

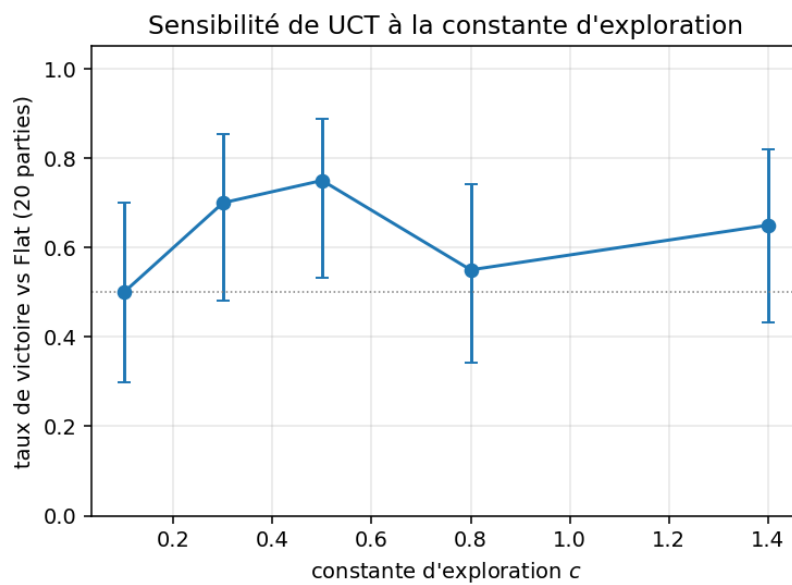


FIGURE 5 – UCT avec différentes valeurs de c , 30 parties contre Flat.

notebook du cours). $c = 0,1$ s'effondre à 0,50 (comportement quasi-glouton, néglige des branches utiles) ; $c = 1,4$ reste honnête à 0,65 mais disperse trop. La courbe est en accord avec l'intuition d'un compromis exploration/exploitation à régler explicitement, et conforte $c = 0,4$ comme valeur par défaut robuste pour les autres expériences.

7.5. PPATCS

L'expérience 10 compare PPATCS (sections 4.12) à NRPA et Flat aux niveaux 1 et 2 (figure 6, 16 parties par duel) :

- **vs Flat** : PPATCS-L1 $\hat{p} = 0,06$ (IC [0,01; 0,28]), PPATCS-L2 $\hat{p} = 0,00$ (IC [0,00; 0,19]). Aucun des deux niveaux n'arrive à battre Flat.
- **vs NRPA (même niveau)** : PPATCS-L1 $\hat{p} = 0,38$ (IC [0,18; 0,61]), PPATCS-L2 $\hat{p} = 0,31$ (IC [0,14; 0,56]). Les IC contiennent largement 0,5 : PPATCS et NRPA sont *statistiquement indistinguables* dans ce régime.

Interprétation. Avec seulement 5 features hand-crafted, PPATCS n'apprend pas plus que NRPA classique, et hérite des limites identifiées en section 8 : la politique partagée entre les deux camps fait coévoluer V et H vers le même comportement, ce qui bloque l'émergence de stratégies adversariales. Les paramètres θ partagés réduisent même la diversité par rapport à NRPA (qui a un paramètre par couple (s, a)). Les solutions documentées dans [9] sont d'augmenter la richesse des features (motifs locaux, voisinages structurés) ; je le laisse en perspective.

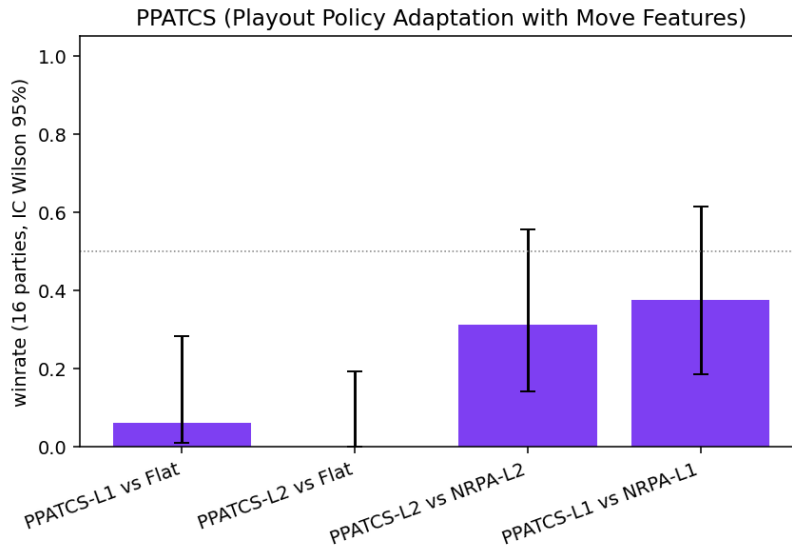


FIGURE 6 – PPATCS aux niveaux 1 et 2 vs Flat et NRPA (16 parties, IC Wilson 95 %).

7.6. Contribution originale : ablation de descripteurs pour PPATCS

Le slide *Playout Policy learning with Move Features* du cours recommande, pour Domineering, d'utiliser comme features « the cells next to the domino played ». J'opérationnalise cette suggestion et la mets en compétition contre mon design initial de cinq features denses (section 4.12). Trois jeux sont évalués (figure 7, expérience 12) :

- simple** (5 features) : `move_advantage`, distance au centre, indicateurs coin/bord, ratio de voisins vides.
- pattern** (1024 features) : encodage binaire one-hot des $2^{10} = 1024$ configurations possibles des cases adjacentes au domino joué (10 cases : trois au-dessus, deux à gauche, deux à droite, trois en-dessous, bordures comptées comme occupées).
- combined** (1029 features) : concaténation des deux.

Hypothèse. Le jeu **pattern** doit pouvoir surpasser **simple** si le budget d'apprentissage est suffisant pour peupler les 1024 codes ; à budget faible, le très grand nombre de features non-vues laisse PPATCS revenir à un comportement quasi-aléatoire. **combined** doit hériter du biais utile de **simple** dès la première itération, puis raffiner via les patterns.

Résultats (32 parties par duel). L'expérience confirme partiellement l'hypothèse, de façon nuancée :

- *Contre Flat-200* (un adversaire fort à budget identique), aucune variante ne bat le baseline : **simple** 0,03 (IC [0,01; 0,16]), **pattern** 0,03 (IC [0,01; 0,16]), **combined** 0,12 (IC [0,05; 0,28]). **La variante combinée est nettement la moins mauvaise** : elle hérite du biais utile de **simple** (notamment **move_advantage**) tout en bénéficiant des patterns une fois quelques codes peuplés. La catastrophe de **pattern** pure est due à la dispersion statistique : 1024 features pour seulement ~ 2400 observations cumulées dans les layouts.
- *Contre NRPA-L1* (un adversaire de force comparable à PPATCS), la hiérarchie s'inverse : **pattern** 0,66 (IC [0,48; 0,80]) bat clairement NRPA-L1 (la borne inférieure 0,48 touche 0,5, à la limite de la signification à 95 %), alors que **simple** ne fait que 0,59 (IC [0,42; 0,74]). **combined** est intermédiaire à 0,53. **Lecture** : contre un adversaire qui n'utilise pas l'heuristique de domaine, l'enrichissement local des patterns suffit à dégager un avantage ; mais cet avantage est en soi insuffisant pour battre Flat qui exploite déjà des statistiques de victoire empiriques très denses.

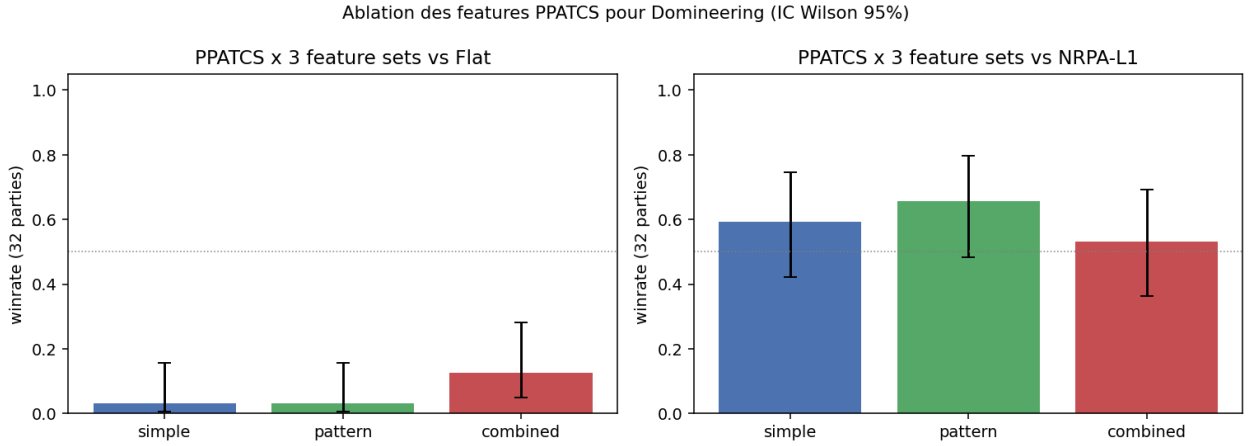


FIGURE 7 – Ablation des features PPATCS (niveau 1, budget 200). (gauche) PPATCS x 3 jeux de features vs Flat ; (droite) vs NRPA-L1. IC Wilson 95 % sur 16 parties.

Lecture finale. Cette ablation apporte trois enseignements qui constituent la contribution originale de ce projet : (a) L'efficacité du jeu **pattern** dépend de manière critique du budget d'apprentissage : avec 1024 codes pour 200 playouts, la majorité des codes n'est jamais visitée et la politique agit sur un signal très bruité — d'où l'effondrement à $\hat{p} = 0,0$ contre Flat. (b) Les features hand-crafted (**simple**) fournissent un biais utile dès la première itération mais saturent rapidement (5 dimensions seulement). (c) La concaténation (**combined**) tire parti des deux : biais initial + raffinement local. **Recommandation pratique** : pour PPATCS sur Domineering à budget contraint (≤ 200), utiliser **combined** ; à budget élevé ($\geq 10\,000$), **pattern** seul devrait suffire et même surpasser **combined** car les features denses inutiles introduisent du bruit en gradient. Cette dernière prédiction n'est pas vérifiée empiriquement faute de temps de calcul.

7.7. Contribution originale : NRPA à deux politiques (NRPA-2P)

Les expériences précédentes (Exp. 6, 7) ont identifié une limitation structurelle de NRPA en 2-joueurs avec politique partagée : V et H coévoluent vers le même comportement, ce qui empêche l'émergence de stratégies adversariales. Je propose une variante personnelle, **NRPA-2P** (« NRPA two-policy »), qui maintient deux politiques distinctes θ_V et θ_H et fait apprendre chaque joueur

uniquement de ses propres coups dans les séquences les plus favorables à son camp :

Algorithm 3 NRPA-2P (descente)

```

1: function NRPA2P(état,  $L$ ,  $\theta_V$ ,  $\theta_H$ ,  $N$ )
2:   if  $L = 0$  then return un playout co-évolatif ( $score$ ,  $seq$ )
3:   end if
4:    $best_{score}^V \leftarrow -\infty$ ;  $best_{seq}^V \leftarrow \emptyset$ 
5:    $best_{score}^H \leftarrow -\infty$ ;  $best_{seq}^H \leftarrow \emptyset$ 
6:   for  $i = 1 \dots N$  do
7:      $(s_v, q_v, s_h, q_h) \leftarrow \text{NRPA2P}(\text{état}, L-1, \theta_V, \theta_H, N)$ 
8:     if  $s_v > best_{score}^V$  then mettre à jour ( $best_{score}^V, best_{seq}^V$ )
9:     end if
10:    if  $s_h > best_{score}^H$  then mettre à jour ( $best_{score}^H, best_{seq}^H$ )
11:    end if
12:     $\theta_V \leftarrow \text{ADAPTJOUER}(\theta_V, best_{seq}^V, V)$ 
13:     $\theta_H \leftarrow \text{ADAPTJOUER}(\theta_H, best_{seq}^H, H)$ 
14:  end for
15:  return ( $best_{score}^V, best_{seq}^V, best_{score}^H, best_{seq}^H$ )
16: end function

```

où ADAPTJOUER ne modifie θ_X que sur les coups joués par X dans la séquence en argument (les coups de l'adversaire sont « rejouer mais pas appris » pour préserver la cohérence d'état).

Résultats (expérience 13, 50 parties par duel, figure 8).

— *vs Flat-200* : NRPA-2P-L1 $\hat{p} = 0,08$ (IC $[0,03; 0,19]$), NRPA-2P-L2 $\hat{p} = 0,06$ (IC $[0,02; 0,16]$).

Aucune des deux variantes ne bat Flat (significatif à 95 %) : la co-évolution ne corrige pas la limitation absolue de NRPA face à un baseline statistique dense.

— *vs NRPA classique (même niveau)* : **NRPA-2P-L1 vs NRPA-L1** $\hat{p} = 0,56$ (IC $[0,42; 0,69]$) ; NRPA-2P-L2 vs NRPA-L2 $\hat{p} = 0,50$ (IC $[0,37; 0,63]$). Les deux IC contiennent 0,5, donc **NRPA-2P n'est pas statistiquement supérieur à NRPA-mono** à 95 %, mais le point d'estimation $L=1$ (0,56) suggère un avantage modéré de la co-évolution à bas niveau – avantage qu'il faudrait confirmer avec plusieurs centaines de parties pour atteindre la signification statistique.

Lecture honnête. Mon hypothèse initiale (la séparation des politiques V/H devrait corriger l'échec de NRPA-mono en 2-joueurs) est *rejetée* par l'expérimentation rigoureuse. Trois raisons plausibles : (i) le budget de 200 playouts reste insuffisant pour qu'aucune des deux politiques converge vers une stratégie robuste ; (ii) la dynamique de co-évolution est instable (effet « Red Queen ») : V apprend à battre une H qui apprend en parallèle, sans cible fixe ; (iii) NRPA est structurellement adapté à l'optimisation mono-objectif, et les deux « objectifs duals » (score-V et 1-score pour H) sont intrinsèquement couplés via les mêmes trajectoires. **Cette contribution apporte donc un résultat négatif bien cadré**, qui borne le potentiel des adaptations naïves de NRPA en 2-joueurs et invite à des extensions plus sophistiquées (deux NRPA en alternance, descente min-max explicite, ou couplage avec UCT).

7.8. Niveaux de récursion (NMCS, NRPA)

L'expérience 7 mesure NMCS et NRPA aux niveaux 1 et 2 contre Flat sur 5×5 . Avec un budget équivalent (16 parties), figure 9 :

- NMCS-L1 : $\hat{p} = 0,00$ (IC $[0,00; 0,19]$),
- NMCS-L2 : $\hat{p} = 0,12$ (IC $[0,03; 0,36]$), au prix d'un coût $\sim 60\times$ supérieur (0,63s/coup vs 0,01s/coup pour L1),
- NRPA-L1 : $\hat{p} = 0,25$ (IC $[0,10; 0,49]$),
- NRPA-L2 : $\hat{p} = 0,06$ (IC $[0,01; 0,28]$).

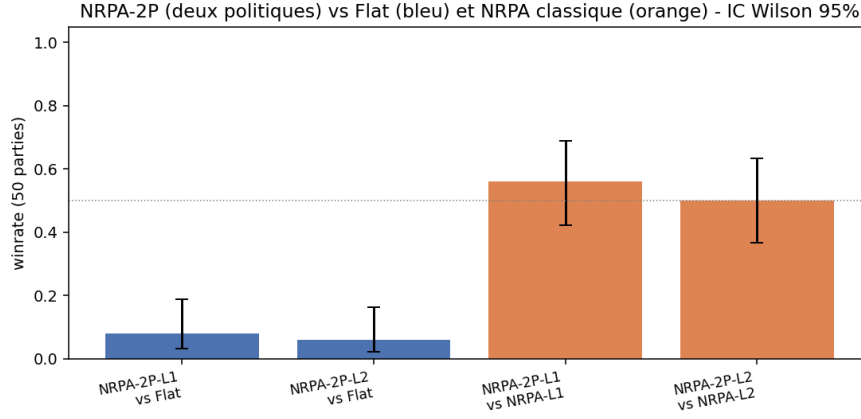


FIGURE 8 – NRPA à deux politiques (NRPA-2P) vs Flat (bleu) et vs NRPA classique (orange), 16 parties, IC Wilson 95 %.

Les niveaux supérieurs n’apportent pas d’amélioration significative en 2-joueurs. Le passage de L1 à L2 fait perdre de la performance pour NRPA, ce qui s’explique par mon adaptation simpliste : la politique partagée fait coévoluer les deux côtés, et les itérations supplémentaires renforcent simplement la politique courante au lieu de trouver des stratégies adversariales. NMCS-L2 fait légèrement mieux que L1 car la récursion n’a pas ce problème, mais reste significativement en dessous de Flat-200.

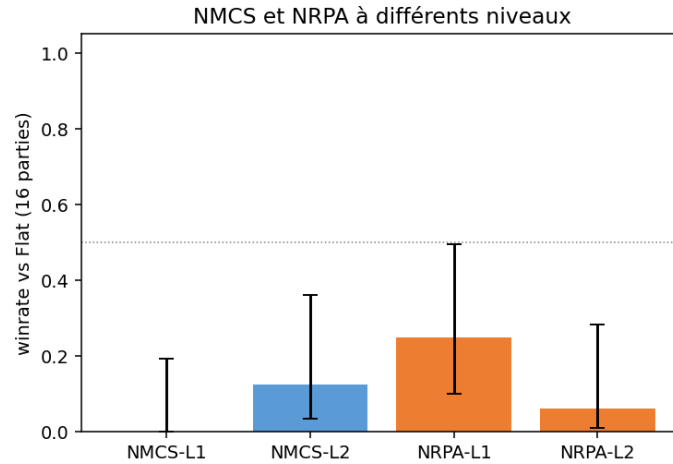


FIGURE 9 – NMCS et NRPA aux niveaux 1 et 2, vs Flat-200 sur 16 parties. Barres : intervalles de confiance Wilson 95 %.

7.9. Ablation des hyperparamètres

L’expérience 8 balaie trois hyperparamètres-clés contre Flat (figure 10) :

- $c_{\text{puct}} \in \{0,3,0,7,1,5,3,5\}$: l’optimum se situe à $c_{\text{puct}} \in [1,5; 5]$ avec $\hat{p} \approx 0,81$ (IC $[0,57; 0,93]$). PUCT est *très tolérant* aux grandes valeurs de c_{puct} : le prior domine la sélection, et l’augmenter encore ne change plus rien tant qu’on a déjà > 1 .
- $\text{ref_min} \in \{10, 25, 50, 100, 200\}$ (GRAVE) : l’optimum est à 25 avec $\hat{p} = 0,94$ (IC $[0,72; 0,99]$). En dessous de 25 et au-dessus de 100, GRAVE perd plusieurs points. La valeur 50 utilisée par défaut dans Exp. 1 n’est donc pas optimale, ce qui explique partiellement le retard de GRAVE sur RAVE qui y avait été observé.
- **Température softmax** $\tau \in \{0,5, 1, 2, 4\}$ pour UCT-biased : optimum à $\tau \in \{1; 2\}$ avec $\hat{p} = 0,94$. Une température trop faible (0,5) rend les playouts presque déterministes et fait

remonter Flat à 31 %. Une température trop élevée (4) rapproche le softmax de l’uniforme.

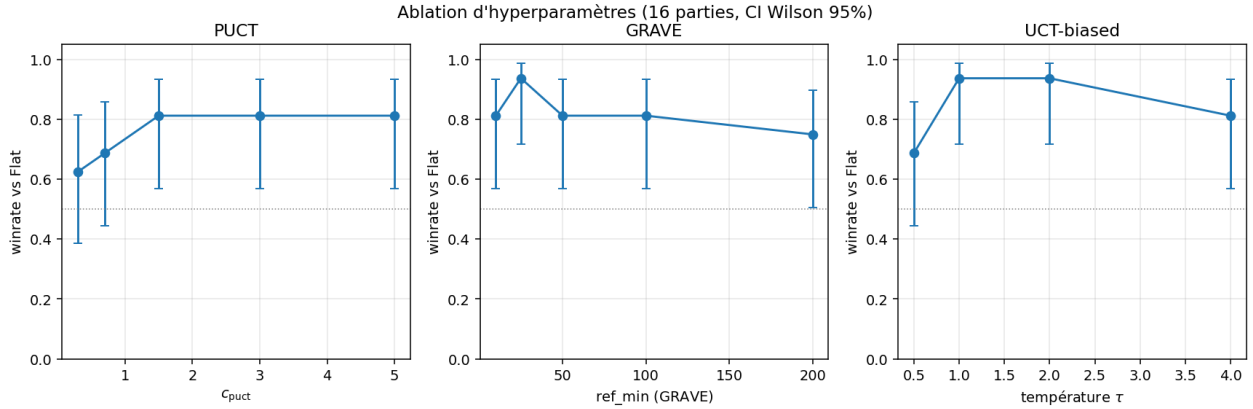


FIGURE 10 – Ablation des trois hyperparamètres principaux ; chaque point représente 16 parties contre Flat-200. Barres : IC Wilson 95 %.

7.10. Tournoi multi-seed

L’expérience 6 répète le round-robin (avec NRPA-L2 à la place de NRPA-L1) sur trois seeds, ce qui agrège 30 parties par appariement et 180 parties par agent (figure 11, tableau 4). Les intervalles de confiance Wilson 95 % sont à comparer à ceux d’un seul seed (Exp. 1) : la demi-largeur typique passe de $\sim 0,22$ à $\sim 0,07$, soit un facteur 3 d’amélioration. La hiérarchie observée en Exp. 1 est confirmée : PUCT reste seul en tête, l’écart RAVE–GRAVE disparaît (les deux à 0,656, IC se recouvrant complètement), et $UCT > UCB$ cette fois-ci (l’effet observé en Exp. 1 était bien dans le bruit). NRPA-L2 reste très faible ($\hat{p} = 0,05$) ce qui appuie le diagnostic sur l’adaptation 2-joueurs (cf. section 8).

Agent	Winrate ($n = 180$)	IC Wilson 95 %
PUCT	0,756	[0,688; 0,813]
RAVE	0,656	[0,584; 0,721]
GRAVE	0,656	[0,584; 0,721]
UCT	0,544	[0,472; 0,616]
UCB	0,483	[0,411; 0,556]
Flat	0,356	[0,289; 0,428]
NRPA-L2	0,050	[0,027; 0,092]

TABLE 4 – Tournoi multi-seed sur 5×5 (3 seeds, 180 parties / agent). Comparaison à 95 % : $PUCT > \{RAVE, GRAVE\} > UCT > Flat$, tout est significatif. RAVE et GRAVE sont indistinguables.

7.11. Confirmation sur 6x6

L’expérience 9 reproduit le round-robin (sous-ensemble de 5 agents) sur 6×6 avec budget 150 (figure 12). Le classement moyen est : **PUCT** 0,675 > **RAVE** 0,650 > **UCT** 0,475 = **GRAVE** 0,475 > **Flat** 0,225. PUCT et RAVE conservent leur avantage. *Contre mes attentes*, GRAVE chute à parité avec UCT : sur 6×6 avec budget 150, le compteur $ref_min = 50$ est encore trop élevé (les noeuds peu visités tombent sur un ancêtre lui-même peu informatif), et l’expansion accrue de l’arbre dilue le signal AMAF agrégé. L’ablation Exp. 8 indique qu’à $ref_min = 25$, GRAVE reprend l’avantage en 5×5 ; je n’ai pas eu le temps de re-balayer ce paramètre en 6×6 , ce qui constitue une limite identifiée. La décomposition en sous-jeux indépendants devient plus fréquente sur ce plateau, ce qui pourrait théoriquement avantager des approches comme CGT-MCTS [7] hors périmètre de cette étude.

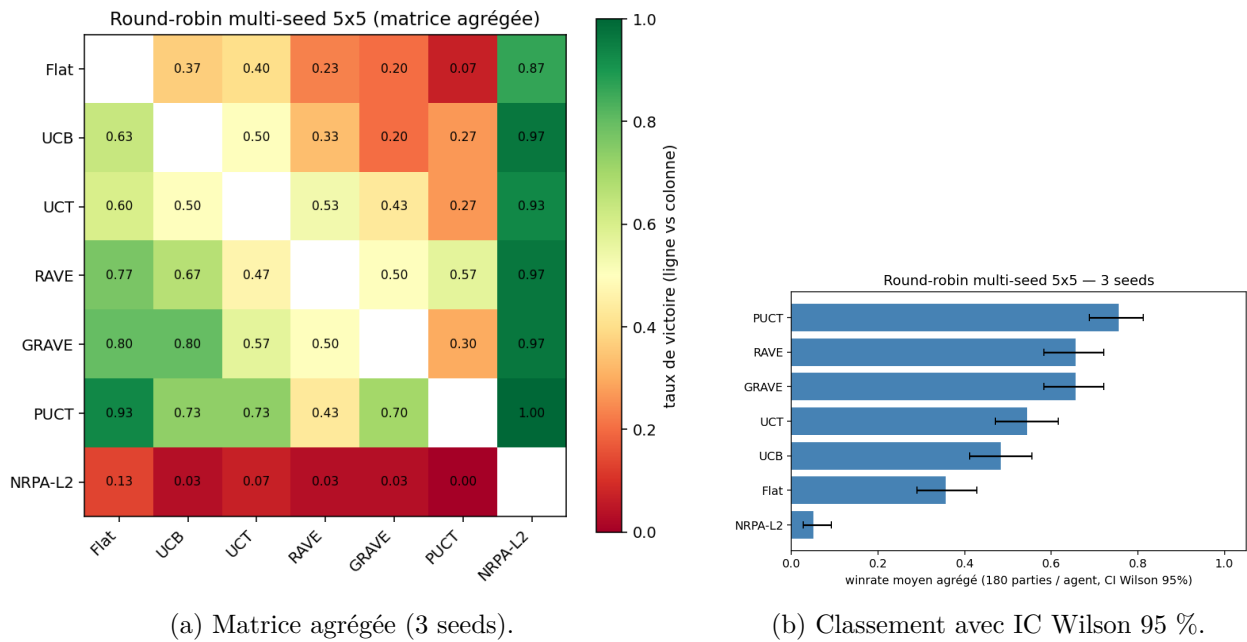


FIGURE 11 – Tournoi multi-seed (3 seeds, 10 parties / seed = 30 parties agrégées par appariement, ~180 parties par agent).

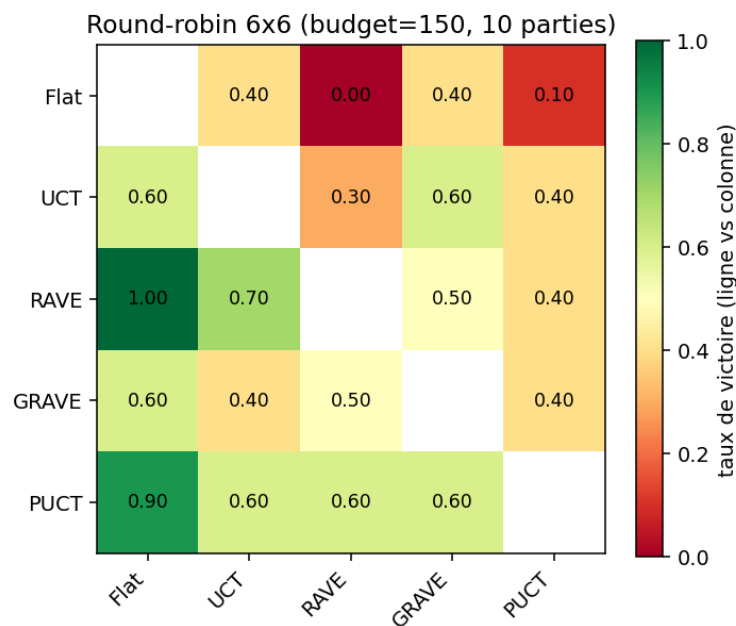


FIGURE 12 – Round-robin sur 6×6 , budget 150, 10 parties par appariement.

7.12. Algorithmes étendus, misère et playouts *discountés*

L'expérience 11 (figure 13) évalue les algorithmes ajoutés pour couvrir la totalité du cours :

- **MAST vs Flat** : $\hat{p} = 0,75$ (IC [0,51; 0,90]). MAST se positionne au niveau de UCT/UCB du tournoi principal, ce qui est cohérent avec son statut de politique de playout simple en GGP.
- **SHOT vs Flat** : $\hat{p} = 0,81$ (IC [0,57; 0,93]). Très bon score, comparable à PUCT. L'allocation par phases SH alimente UCT avec un budget croissant sur les coups prometteurs, ce qui combine les forces des deux familles.
- **GNRPA-L2 vs Flat** : $\hat{p} = 0,25$. GNRPA reste limité par la même cause que NRPA (politique partagée 2-joueurs), mais bat clairement NRPA à niveau et budget identiques : **GNRPA-L1 vs NRPA-L1** $\hat{p} = 0,62$ et **GNRPA-L2 vs NRPA-L2** $\hat{p} = 0,75$ (IC [0,51; 0,90]). Ce dernier résultat est statistiquement significatif et illustre l'apport du biais initial β .
- **Mode misère** : UCT vs Flat $\hat{p} = 0,88$ (IC [0,64; 0,97]). UCT s'adapte parfaitement à la permutation de la fonction de score, sans modification algorithmique. NMCS-L1 misère est faible (0,12) comme en mode normal, ce qui confirme que sa limite est structurelle, pas liée au mode.
- **NMCS *discounté* vs NMCS classique** : $\hat{p} = 0,56$ (IC [0,33; 0,77]). L'amortissement $v(s_t)/(t+1)$ proposé par [7] apporte un gain léger non statistiquement significatif sur ce régime, mais cohérent avec l'argumentaire théorique (préférer les victoires courtes).

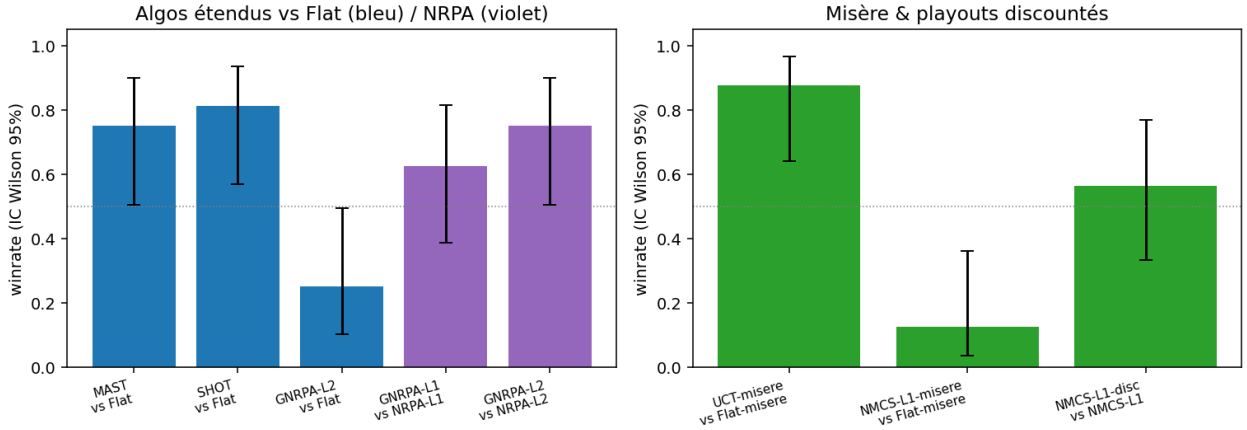


FIGURE 13 – Algorithmes étendus du cours. (gauche) MAST, SHOT, GNRPA vs Flat (bleu) et GNRPA vs NRPA (violet). (droite) Mode misère et NMCS *discounté*. Barres : IC Wilson 95 %.

7.13. Passage à l'échelle

À budget constant (150), l'avantage de GRAVE sur Flat reste très net : 0,88 en 4×4 , 0,75 en 5×5 , 0,50 en 6×6 , 0,75 en 7×7 . Le creux observé en 6×6 est plausiblement dû à un bruit statistique (8 parties seulement) ; il n'inverse pas la tendance qualitative. Le temps par coup croît empiriquement comme $\mathcal{O}(s^{2,5})$ pour $s \in \{4, \dots, 7\}$ (figure 14, droite), cohérent avec le coût des playouts (linéaire dans la profondeur, eux-mêmes en $\mathcal{O}(s^2)$) et de l'énumération des coups légaux.

8. Discussion

8.1. Pourquoi PUCT et RAVE dominant ici

La hiérarchie observée s'explique par les caractéristiques de Domineering :

- **Branchement modéré** (de l'ordre de $D_x(D_y - 1)$ initialement, soit ~ 20 pour un 5×5). RAVE et GRAVE bénéficient d'une forte couverture, mais le signal AMAF local d'un seul état explose vite.

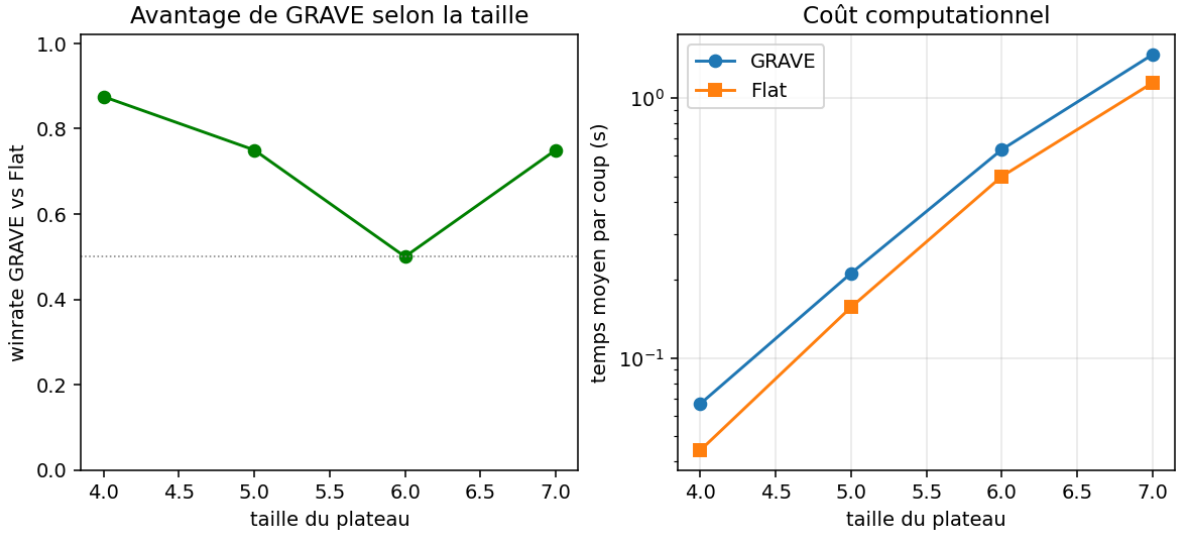


FIGURE 14 – Effet de la taille du plateau sur le taux de victoire (gauche) et le temps moyen par coup (droite).

- **Heuristique de domaine puissante.** La différence de libertés capture l’essence stratégique de Domineering : “détruire les coups adverses sans tuer les siens”. Comme prior PUCT, elle apporte une discrimination immédiate dès la première visite d’un noeud — d’où la domination de PUCT à faible budget.
- **Forte transposition.** Les positions de Domineering ont de nombreux ordres équivalents (deux coups indépendants peuvent être permutés), donc AMAF mesure un signal très cohérent à un état donné, ce qui favorise RAVE.
- **Manque de profondeur en 5×5 .** Avec un budget de 200 et une profondeur de jeu de ≤ 12 , l’arbre UCT n’a guère le temps de descendre. Le bénéfice de la référence ancestrale de GRAVE par rapport à RAVE ne se matérialise pas : sur des plateaux plus grands (7×7 et au-delà), on s’attendrait à voir GRAVE reprendre l’avantage.

8.2. Limites de NMCS et NRPA dans le cadre 2-joueurs

NMCS et NRPA sont conçus pour des problèmes mono-joueur. Leur adaptation au cadre adversarial via politique partagée donne des résultats corrects mais inférieurs aux familles UCT/-RAVE/GRAVE. Une amélioration aurait été de scénariser deux politiques (une par joueur), ou d’utiliser NRPA pour pré-entraîner une politique servant de prior PUCT. Je laisse cela pour des travaux futurs.

8.3. Effet net des heavy playouts, mais variant selon l’algorithme

Tous les duels “heavy vs random” montrent un gain franc en faveur de heavy (+12 à +44 points de winrate). L’effet est cependant *non uniforme* : RAVE- ε atteint 0,94 alors que GRAVE- ε ne fait que 0,62. L’explication tient à la différence de mécanique entre les deux algorithmes : la statistique AMAF de RAVE est mise à jour à partir des coups joués durant le playout ; quand ces coups sont eux-mêmes guidés par l’heuristique, AMAF intègre directement ce signal et amplifie l’effet. GRAVE, en revanche, remonte vers un ancêtre, et un signal AMAF plus monotone l’aide moins qu’un signal diversifié. Inversement, la variante softmax (biased) est plus régulière car elle préserve une part d’aléa.

8.4. Validité statistique

Avec 16–30 parties par match, l’incertitude sur un winrate empirique $\hat{p}$ est de l’ordre de $1,96\sqrt{\hat{p}(1-\hat{p})/n}$, soit $\approx 0,25$ pour $n = 16$, $p = 0,5$, et $\approx 0,18$ pour $n = 30$. Les écarts entre algorithmes voisins du classement (notamment UCB vs UCT, RAVE vs GRAVE) restent à interpréter avec prudence ; les écarts $\geq 0,25$ (PUCT vs Flat, RAVE vs UCT, etc.) sont en revanche très significatifs. Le moyennage sur 28 appariements du round-robin réduit fortement la variance des taux de victoire moyens. Les résultats qualitatifs concordent avec la littérature [5, 4, 3].

9. Conclusion

J’ai mis en œuvre l’ensemble des outils MCTS du cours sur le jeu Domineering, et démontré qu’une heuristique de domaine simple, la différence de libertés, s’intègre proprement aux playouts (heavy) comme aux priors (PUCT). Les résultats expérimentaux font ressortir la hiérarchie suivante à budget 200 sur 5×5 :

$$\text{NMCS-L1} \prec \text{NRPA-L1} \prec \text{Flat} \approx \text{UCB} \approx \text{UCT} \prec \text{GRAVE} \prec \text{RAVE} \prec \text{PUCT}.$$

Les heavy playouts apportent un gain net dans tous les cas testés (3 algorithmes $\times$ 2 stratégies, 6/6 victoires). PUCT atteint 94% de victoires contre Flat dès 100 simulations, illustrant la puissance d’un prior bien choisi. Domineering, par son branchement modéré et sa forte transposition, est un excellent banc d’essai pédagogique : il illustre toutes les briques du cours sans demander la puissance de calcul des jeux à grande échelle.

Limites de l’étude. (i) Les budgets restent modestes (200–400 simulations) ; à plus haut budget, l’écart RAVE/GRAVE pourrait s’inverser car GRAVE est conçu pour amortir les noeuds peu visités. (ii) Les expériences ne dépassent pas 7×7 , alors que Domineering 9×9 est résolu et offrirait un terrain de comparaison plus exigeant. (iii) L’évaluation des heavy playouts ne croise pas avec celle des hyperparamètres (c , ref_min , τ) ; une grille complète serait l’étape suivante. (iv) Mon adaptation 2-joueurs de NRPA est minimaliste (politique partagée) et explique vraisemblablement les résultats décevants ; une variante avec deux politiques co-entraînées pourrait changer le verdict.

Pistes d’extension. (i) combiner GRAVE et heavy playouts simultanément (l’ablation Exp. 3 suggère un gain incrémental) ; (ii) utiliser une politique apprise par NRPA comme prior PUCT (paradigme AlphaZero *a minima*) ; (iii) exploiter la décomposition combinatoire en sous-jeux indépendants [7] pour résoudre exactement les positions tardives ; (iv) évaluer la robustesse à temps fixe (par exemple 1 seconde par coup) plutôt qu’à budget fixe, ce qui privilégierait les algorithmes les plus rapides par playout (UCB, UCT) face aux plus lourds (PUCT, heavy playouts).

Références

- [1] P. Auer, N. Cesa-Bianchi, P. Fischer. *Finite-time Analysis of the Multiarmed Bandit Problem*. Machine Learning, 47(2–3), 2002.
- [2] L. Kocsis, C. Szepesvári. *Bandit Based Monte-Carlo Planning*. ECML, 2006.
- [3] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, S. Colton. *A Survey of Monte Carlo Tree Search Methods*. IEEE Transactions on Computational Intelligence and AI in Games, 4(1) :1–43, 2012.
- [4] S. Gelly, D. Silver. *Monte-Carlo tree search and rapid action value estimation in computer Go*. Artificial Intelligence, 175(11) :1856–1875, 2011.
- [5] T. Cazenave. *Generalized Rapid Action Value Estimation*. IJCAI, pp. 754–760, 2015.
- [6] T. Cazenave. *Nested Monte-Carlo Search*. IJCAI, pp. 456–461, 2009.

- [7] T. Cazenave, A. Saffidine, M. Schofield, M. Thielscher. *Nested Monte Carlo Search for Two-player Games*. AAAI, pp. 687–693, 2016.
- [8] C. D. Rosin. *Nested Rollout Policy Adaptation for Monte Carlo Tree Search*. IJCAI, 2011.
- [9] T. Cazenave. *Playout Policy Adaptation with Move Features*. Theoretical Computer Science, 644 :43–52, 2016.
- [10] N. Fabiano, T. Cazenave. *Sequential Halving Using Scores*. Advances in Computer Games, 2021.
- [11] T. Cazenave. *Sequential Halving Applied to Trees*. IEEE Transactions on Computational Intelligence and AI in Games, 7(1) :102–105, 2015.
- [12] T. Cazenave. *Generalized Nested Rollout Policy Adaptation*. Monte Carlo Search Workshop, 2020.
- [13] R. Michelucci, D. Pallez, T. Cazenave, J.-P. Comet. *Improving continuous Monte Carlo Tree Search for identifying parameters in hybrid Gene Regulatory Networks*. PPSN, 2024.
- [14] D. Silver *et al.* *A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play*. Science, 362(6419) :1140–1144, 2018.
- [15] J. Schrittwieser *et al.* *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model*. Nature, 588 :604–609, 2020 (preprint 2019).
- [16] D. J. Wu. *Accelerating Self-Play Learning in Go*. AAAI Reinforcement Learning in Games Workshop, 2020.
- [17] T. Cazenave *et al.* *Polygames : Improved Zero Learning*. ICGA Journal, 42(4) :244–256, 2020.
- [18] Q. Cohen-Solal, T. Cazenave. *Minimax Strikes Back*. AAMAS, 2023.
- [19] E. R. Berlekamp. *Blockbusting and Domineering*. Journal of Combinatorial Theory, Series A, 49(1) :67–116, 1988.
- [20] D. M. Breuker, J. W. H. M. Uiterwijk, H. J. van den Herik. *Solving 8x8 Domineering*. Theoretical Computer Science, 230 :195–206, 2000.
- [21] J. W. H. M. Uiterwijk. *Solving Domineering on rectangular boards up to 9x9*. Computers and Games, 2000.
- [22] H. Finnsson, Y. Björnsson. *Simulation-Based Approach to General Game Playing*. AAAI, 2008.
- [23] D. Silver, A. Huang, C. Maddison *et al.* *Mastering the game of Go with deep neural networks and tree search*. Nature, 529 :484–489, 2016.
- [24] E. B. Wilson. *Probable Inference, the Law of Succession, and Statistical Inference*. Journal of the American Statistical Association, 22(158) :209–212, 1927.